
Non-Linear Filtering for Target Tracking

Mathematical Foundations of the IMM-CKF Architecture

Huiyuan Zang

Advanced Concepts, EDGE Group

First Edition

June 2026

Copyright © 2026 Huiyuan Zang
All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning, or otherwise, without the prior written permission of the author.

First Edition: June 2026

Contents

Preface	vii
Acknowledgments	ix
Part I: The Mathematical Engine Room	1
1 Advanced Calculus & Optimization	2
1.1 Partial Derivatives and the Jacobian Matrix	2
1.2 Numerical Integration and Cubature Rules	3
1.3 Multivariate Taylor Series Expansion	4
2 Linear Algebra for Systems	6
2.1 Vectors, matrices, and tensor basics.	6
2.2 Eigenvalues, eigenvectors, and matrix decompositions (Cholesky decomposition, Singular Value Decomposition).	6
2.3 Matrix inversion lemma (Woodbury identity) and proofs.	8
3 Differential Equations & Kinematics	9
3.1 Ordinary Differential Equations (ODEs) and continuous-time system modeling.	9
3.2 Discretization of continuous-time models (Runge-Kutta methods).	10
3.3 Modeling constant velocity (CV), constant acceleration (CA), and coordinated turn (CT) dynamics.	11
3.3.1 The Constant Velocity (CV) Model	11
3.3.2 The Constant Acceleration (CA) Model	13
3.3.3 The Coordinated Turn (CT) Model	15
4 Probability Theory & Stochastic Processes	18
4.1 Random variables, probability density functions (PDFs), and Bayes' Theorem.	18
4.2 Gaussian distributions and their properties (multivariate normal distribution).	18
4.3 Markov processes and transition probabilities.	18
Part II: Dynamic Systems and The Optimal Estimator	19
5 Control System Fundamentals	20
5.1 State-space representation of dynamic systems.	20
5.2 Observability and controllability.	20
5.3 Process noise vs. measurement noise.	20
6 The Bayesian Filtering Framework	21
6.1 The recursive Bayes filter: Prediction and Update steps.	21

6.2 Why closed-form solutions are rare (the integration problem).	21
Part III: The Classic Standard Kalman Filter (KF)	22
7 Derivation of the Standard KF	23
7.1 Mathematical proof of the linear Kalman Filter from Bayesian principles.	23
7.2 The Minimum Mean Square Error (MMSE) estimator.	23
8 Tuning and Diagnostics	24
8.1 Covariance matching and innovation analysis.	24
8.2 Filter divergence and numerical stability considerations.	24
Part IV: Conquering Non-Linearity	25
9 The Extended Kalman Filter (EKF)	26
9.1 Linearizing non-linear functions using the Jacobian.	26
9.2 Mathematical proof and limitations of the EKF (truncation errors).	26
10 The Unscented Kalman Filter (UKF)	27
10.1 The Unscented Transform (UT): Choosing sigma points.	27
10.2 Proof of how UT captures mean and covariance better than linearization.	27
11 The Cubature Kalman Filter (CKF)	28
11.1 The spherical-radial cubature rule.	28
11.2 Mathematical derivation of cubature points.	28
11.3 Comparative analysis: CKF vs. UKF in high-dimensional state spaces.	28
Part V: Maneuvering Targets and Multiple Models	29
12 Interacting Multiple Model (IMM) Estimation	30
12.1 The challenge of maneuvering target tracking.	30
12.2 Markov chain mixing of model probabilities.	30
12.3 The IMM architecture: Interaction, Filtering, Model Probability Update, and Combination.	30
13 The Synthesis: IMM-CKF	31
13.1 Embedding the Cubature Kalman Filter within the IMM framework.	31
13.2 Handling highly non-linear maneuvers (e.g., transitioning from a linear trajectory to a sharp, high-G coordinated turn).	31
13.3 Complete algorithm walkthrough and mathematical proof of convergence	31
Part VI: Architecture and Implementation	32
14 Software Architecture for State Estimation	33
14.1 Object-oriented design patterns for filter development (e.g., in modern C++).	33
14.2 Memory management and real-time processing constraints.	33
15 Hardware Deployment Considerations	34
15.1 Deploying complex estimators on embedded systems.	34
15.2 Exploiting parallelization for matrix operations.	34
16 Bibliography	35
16.1 Articles	35

16.2 Books	35
Bibliography	36
Index	37

List of Tables

List of Figures

Preface

The Kalman Filter (KF) is arguably the most important algorithm in modern estimation theory. At its core, it is an elegant mathematical framework designed to extract the true state of a system from a series of incomplete and noisy measurements. In a perfectly linear world with Gaussian noise, the standard KF is the optimal estimator.

However, modern tactical tracking systems do not operate in a linear world.

In Real-World Vision-and-Telemetry-Based application, tracking has traditionally relied on 2D pixel-space bounding boxes generated by visual sensors. While sufficient for rudimentary camera-following, a 2D bounding box is physically meaningless for a real-world multi-sensor fusion system because pixels lack physical scale. A drone at 100 meters and an airliner at 10,000 meters can occupy the exact same number of pixels. To estimate actual target kinematics—true velocity, physical size, and metric trajectory—the tracking system must project 2D detections into a 3D Earth-fixed coordinate system, such as a North-East-Down (NED) frame.

This projection is where textbook mathematics collide with engineering reality. Mapping a 2D pixel backwards through nested Euler rotation matrices (representing the Gimbal, Mount, and UAV chassis) and a pinhole camera's perspective divide introduces severe mathematical nonlinearities.

The Evolution of Non-Linear Filtering

To handle systems where the measurement function is non-linear, engineers traditionally turn to variations of the Kalman Filter, though each comes with distinct failure modes in high-stress environments:

- **The Extended Kalman Filter (EKF):** The EKF attempts to linearize the system by calculating the Jacobian (the first-order derivative) of the measurement function. However, deriving the analytical Jacobian of nested trigonometric rotation matrices coupled with perspective division is computationally fragile. During high-agility target maneuvers, the EKF's Taylor series approximation frequently fails, leading to rapid linearization error and track divergence.
- **The Unscented Kalman Filter (UKF):** The UKF avoids Jacobians by propagating a deterministic set of sigma points through the non-linear function. While an improvement, the UKF requires the manual tuning of scaling parameters. In higher-dimensional state spaces subjected to extreme measurement noise, the central sigma point can develop a negative weight. This frequently causes the state covariance matrix to lose positive definiteness, resulting in a catastrophic numerical crash (NaN output) on edge hardware.
- **The Cubature Kalman Filter (CKF):** The CKF resolves these issues by utilizing a third-degree spherical-radial cubature rule. It requires zero manual tuning parameters and mathematically guarantees strictly positive weights for all propagated points. This ensures maximum numerical stability, making it the premier choice for visual coordinate projection.

The Need for the IMM Architecture

Even with a stable non-linear filter, a single kinematic model cannot track all target types. For instance, a tracker using a purely 3D model on a ground target will misinterpret inherent bounding-box pixel jitter as vertical velocity, causing the mathematical track to “drift” into the sky or sink underground. Conversely, a constrained 2D ground model will completely fail to track an airborne target.

Furthermore, modern visual AI sensors act as noise multipliers. Bounding boxes inherently vibrate frame-to-frame. If an engineer implements a standard 11-state Constant Acceleration (CA) model, this high-frequency pixel jitter is mathematically interpreted as violent, instantaneous acceleration. The filter amplifies this noise, causing the predicted target state to vibrate uncontrollably until the track is completely lost.

The Interacting Multiple Model (IMM) architecture solves this by running multiple discrete mathematical models in parallel (e.g., a Ground model enforcing a strict altitude constraint, and an Airborne Bearings-Only model). The IMM dynamically shifts probability weights between these models based on how accurately their internal predictions match the incoming visual measurements.

Engineering Example 1: The Domain Transition (Liftoff)

When a drone is stationary on the tarmac, the IMM assigns maximum probability weight to the constrained Ground model. The moment the drone takes off, the 2D bounding box moves vertically in the video frame. The Ground model, mathematically insisting the altitude is zero, registers a massive prediction error (Innovation), and its calculated likelihood collapses. Simultaneously, the Airborne model correctly predicts the vertical movement. Governed by a Markov transition matrix, the IMM instantly transfers the tracking weight to the Airborne model, seamlessly unlatching the target from the Earth without losing the tracking ID.

Engineering Example 2: Multispectral Sensor Toggling

Consider an interceptor toggling its payload from a high-resolution 1080p visible sensor to a 480p Infrared (IR) sensor. The physical target kinematics do not change, but the measurement uncertainty space is drastically altered. IR microbolometers suffer from “thermal blooming,” where thermal inertia causes the heat signature of a fast-moving target to smear across pixels, resulting in severe bounding box fluctuation. To prevent the tracking filter from chasing this noise—which would cause mechanical whiplash in the gimbal—the IMM-CKF dynamically injects an inflated measurement covariance scalar during the hardware toggle, forcing the filter to momentarily distrust the visual edges and rely on its internal kinematic predictions.

This book bridges the gap between pure mathematics and these operational realities. We will build the IMM-CKF from the ground up, proving every equation, and translating those proofs into robust C++ architectures designed for strict Size, Weight, and Power (SWaP) constrained hardware.

Acknowledgments

Writing a book that bridges the gap between pure mathematical theory and strict engineering reality is not a solitary endeavor. It is built upon the shoulders of giants and refined through the patience of peers and family.

First, my deepest gratitude goes to the mathematicians and engineers who laid the foundation for modern state estimation. From Rudolf E. Kálmán's original masterpiece, to the pioneers of non-linear filtering, and the architects of the cubature rule—thank you. Your theoretical brilliance has allowed generations of engineers to pull order from chaos. I also want to thank the wider aerospace and software engineering community, whose relentless pursuit of robust tracking systems under extreme constraints inspired the pragmatic approach of this book.

I am immensely grateful to the reviewers, colleagues, and early readers who dedicated their time to comb through the manuscript. Translating high-level mathematics into production C++ code is a complex task, and your rigorous feedback, mathematical debugging, and code reviews were instrumental in catching errors and refining the proofs. Your dedication has made this a far more accurate and valuable guide. Any errors that remain are entirely my own.

Finally, and most importantly, I want to thank my family. To my wife, Jinjing: thank you for your unwavering support throughout this demanding process. You constantly encourage me, ground me, and push me to be a better person every single day. And to my daughter, Halley: you are the angel of my life. Your bright spirit and joy enlighten my world, reminding me of the true purpose behind the work we do. This book is as much yours as it is mine.

Part I: The Mathematical Engine Room

Author's Note: Building the Toolbox

If you are an engineer looking at the integrals, partial derivatives, and series expansions in the upcoming pages and feeling a sense of mathematical dread, take a breath. This first section of the book exists simply to establish your foundational “toolbox.”

You do not need to perfectly memorize how to analytically solve a multivariate Jacobian or calculate a spherical-radial integral right now. In the later chapters on the Extended (EKF), Unscented (UKF), and Cubature Kalman Filters (CKF), we will pull these tools out one by one. When we do, we will provide complete, step-by-step mathematical proofs explaining exactly how and why they are used to process real sensor data, alongside the C++ architectures to implement them on edge hardware.

For now, as you read through these mathematical prerequisites, simply focus on understanding the fundamental purpose of each tool and why a tracking system might need it.

Chapter 1

Advanced Calculus & Optimization

Before a tracking system can estimate a target's state, it must understand how that state changes over time and how it relates to the sensors observing it. In linear systems, these relationships are simple algebraic multipliers. In the real world of visual sensors and maneuvering targets, these relationships are highly non-linear, requiring advanced calculus to break them down into computable components.

1.1 Partial Derivatives and the Jacobian Matrix

The Mathematical Concept

For a single-variable function $y = f(x)$, the derivative $f'(x)$ gives the rate of change. However, tracking filters operate on multi-dimensional state vectors (e.g., 3D position, velocity, and acceleration). For a multi-variable vector function $\mathbf{y} = \mathbf{h}(\mathbf{x})$, where $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^m$, we must calculate the partial derivative of every output with respect to every input.

This forms the **Jacobian Matrix** (J):

$$J = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \frac{\partial h_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_m}{\partial x_1} & \cdots & \frac{\partial h_m}{\partial x_n} \end{bmatrix}$$

Application in Target Tracking (The EKF)

The Extended Kalman Filter (EKF) relies entirely on the Jacobian matrix. When a camera observes a drone, the measurement function $\mathbf{z} = \mathbf{h}(\mathbf{x})$ maps the target's 3D real-world coordinates into 2D pixel space. Because this involves perspective division and trigonometric rotations, it is highly non-linear. The EKF uses the Jacobian $H_k = \frac{\partial \mathbf{h}}{\partial \mathbf{x}}$ evaluated at the current state estimate to linearize the function, allowing the filter to project the state covariance matrix into the measurement space[2]:

$$S_k = H_k P_k H_k^T + R_k$$

The Engineering Dilemma

Deriving the analytical Jacobian for a nested camera projection matrix (UAV chassis $\rightarrow$ gimbal $\rightarrow$ camera pinhole) results in massive, computationally fragile trigonometric chains. On SWaP-constrained

edge devices, evaluating these long sine/cosine chains for every target, every frame, burns critical CPU cycles. Furthermore, if the target maneuvers rapidly, the first-order Taylor series approximation of the Jacobian fails, causing catastrophic linearization error and track divergence.

1.2 Numerical Integration and Cubature Rules

The Mathematical Concept

Because analytical Jacobians are dangerous in high-agility tracking, modern filtering seeks to avoid them entirely. Instead of linearizing the function, we want to calculate the true expected value of the non-linear function mathematically. This requires evaluating a Gaussian-weighted integral over the entire state space:

$$I(f) = \int_{\mathbb{R}^n} f(\mathbf{x}) \exp(-\mathbf{x}^T \mathbf{x}) d\mathbf{x}$$

Solving this integral analytically in real-time is impossible. The **Spherical-Radial Cubature** Rule provides a numerical approximation. It decomposes the multi-dimensional integral into a spherical integral (direction) and a radial integral (distance) [1]. By carefully selecting exactly $2n$ deterministic points (where n is the state dimension), we can approximate the true integral with third-degree accuracy [1].

Application in Target Tracking (The CKF)

This integral is the beating heart of the Cubature Kalman Filter (CKF). Instead of differentiating the camera projection model, the CKF generates a set of 3D points representing the current uncertainty, pushes those specific points through the true non-linear camera projection model, and calculates the variance of the output.

C++ Implementation: Cubature Point Generation

In production code, generating these points must be done without dynamic memory allocation to ensure deterministic execution times in the main tracking loop. We utilize the Eigen library, a staple in high-performance C++ engineering.

```
#include <Eigen/Dense>
#include <vector>
#include <cmath>

using namespace Eigen;

class CubatureMath {
public:
    // Generates 2n cubature points based on the state dimension (n)
    // x: Current state vector
    // P_sqrt: The lower triangular matrix from Cholesky decomposition of Covariance P
    static MatrixXd generateCubaturePoints(const VectorXd& x, const MatrixXd& P_sqrt) {
        int n = x.rows();
        int num_points = 2 * n;
```

```

// Pre-allocate the matrix to avoid heap fragmentation in the main loop
MatrixXd cubaturePoints(n, num_points);

// The scaling factor for the spherical-radial rule is simply sqrt(n)
double scale = std::sqrt(static_cast<double>(n));

for (int i = 0; i < n; ++i) {
    // Positive axis points
    cubaturePoints.col(i) = x + scale * P_sqrt.col(i);
    // Negative axis points
    cubaturePoints.col(i + n) = x - scale * P_sqrt.col(i);
}

return cubaturePoints;
}
};

```

1.3 Multivariate Taylor Series Expansion

The Mathematical Concept

In engineering mathematics, it is often easier to approximate a highly complex function rather than solve it perfectly. A Taylor series allows us to approximate any smooth non-linear function as an infinite sum of polynomial terms, based on its derivatives at a specific operating point. For a multi-dimensional state vector $\mathbf{x}$ evaluated near a known predicted state $\hat{\mathbf{x}}$, the multivariate Taylor series expansion of a non-linear measurement function $h(\mathbf{x})$ is expressed as:

$$h(\mathbf{x}) = h(\hat{\mathbf{x}}) + J(\mathbf{x} - \hat{\mathbf{x}}) + \frac{1}{2!}(\mathbf{x} - \hat{\mathbf{x}})^T \mathcal{H}(\mathbf{x} - \hat{\mathbf{x}}) + \dots$$

Where: - J is the Jacobian matrix (the first-order derivative, representing the immediate linear slope).
- $\mathcal{H}$ is the Hessian matrix (the second-order derivative, representing the curvature).
- The $+$... represents an infinite series of increasingly higher-order derivatives.

Application in Target Tracking (The EKF Linearization)

In classic tracking, we absolutely require matrices to calculate and propagate covariance (uncertainty). The Taylor series is the mathematical bridge that attempts to force a non-linear world to behave linearly.

The Extended Kalman Filter (EKF) takes a harsh, pragmatic approach to the equation above: it truncates the Taylor series right after the first-order Jacobian term, completely discarding the Hessian and all higher-order derivatives. It assumes that $h(\mathbf{x}) \approx h(\hat{\mathbf{x}}) + J(\mathbf{x} - \hat{\mathbf{x}})$.

The Engineering Dilemma

This first-order Taylor series approximation only holds mathematically true if the actual target state $\mathbf{x}$ remains extremely close to our filter's prediction $\hat{\mathbf{x}}$. If a fixed-wing UAV is flying straight and level, this approximation works well enough.

However, if an evasive drone suddenly executes a violent, high-G split-S maneuver, the true spatial state diverges rapidly from the filter's linear prediction. As the distance between x and $\hat{x}$ grows, the ignored higher-order terms (the truncation errors) become massive. The EKF's first-order Taylor series approximation mathematically shatters, causing rapid linearization error and total track divergence. This fundamental geometric flaw is precisely why modern aerospace interceptors must abandon the EKF and migrate toward the UKF and CKF architectures, which do not rely on Taylor series truncation.

C++ Implementation: The Cost of Linearization

In an object-oriented tracking architecture, understanding the difference between the true non-linear evaluation and the Taylor series approximation dictates how you architect your measurement updates.

```
#include <Eigen/Dense>
#include <cmath>

using namespace Eigen;

class KinematicApproximation {
public:
    // 1. The True Non-Linear Measurement (e.g., Camera Projection)
    // In the CKF and UKF, we pass deterministic sigma/cubature points directly through this,
    // preserving the true curvature of the physics.
    static VectorXd trueNonLinearMeasurement(const VectorXd& x) {
        VectorXd z(2);
        // Example: highly non-linear perspective division and trigonometric rotation
        z(0) = std::sin(x(0)) / x(2);
        z(1) = std::cos(x(1)) / x(2);
        return z;
    }

    // 2. The First-Order Taylor Approximation (The EKF Approach)
    //  $h(x) \approx h(x_{\text{Hat}}) + J * (x - x_{\text{Hat}})$ 
    static VectorXd linearApproximation(const VectorXd& xTrue, const VectorXd& xHat, const MatrixXd& Jacobian) {
        VectorXd zHat = trueNonLinearMeasurement(xHat);
        VectorXd deltaX = xTrue - xHat;

        // The EKF applies the Jacobian but completely ignores second-order
        // curvature (the Hessian), leading to catastrophic errors during rapid maneuvers.
        return zHat + (Jacobian * deltaX);
    }
};
```

Chapter 2

Linear Algebra for Systems

Tracking algorithms are fundamentally exercises in linear algebra. Every prediction, measurement, and update is a matrix operation. To deploy an IMM-CKF on a modern embedded architecture (such as an NVIDIA Jetson Orin NX), an engineer must not only understand the mathematical identities but also how matrix shapes and memory bandwidth impact the overall processing pipeline.

2.1 Vectors, matrices, and tensor basics.

The Mathematical Concept

A tracking system defines the physical world using column vectors. A state vector $\mathbf{x} \in \mathbb{R}^n$ holds the target's kinematics (e.g., X, Y, Z positions and velocities). Matrices act as operators that transform these vectors. The transition matrix $F \in \mathbb{R}^{n \times n}$ projects the state forward in time, while the measurement matrix $H \in \mathbb{R}^{m \times n}$ maps the n -dimensional state space into an m -dimensional observation space.

Application in Target Tracking

The dimensionality of these vectors and matrices dictates the computational load of the filter. An 11-state Constant Acceleration (CA) model requires maintaining an 11×11 covariance matrix containing 121 elements. An 8-state Constant Velocity (CV) model requires an 8×8 matrix with 64 elements.

While theoretical texts freely expand state dimensions, embedded systems processing multispectral perception pipelines face strict limits on Tensor Core availability. Structuring matrices cleanly ensures cache-friendly memory access during high-frequency telemetry fusion.

2.2 Eigenvalues, eigenvectors, and matrix decompositions (Cholesky decomposition, Singular Value Decomposition).

The Mathematical Concept

In Bayesian filtering, uncertainty is modeled using the State Covariance Matrix, P . A fundamental rule of probability is that variance cannot be negative. Therefore, a valid covariance matrix must be Positive Definite. Mathematically, a symmetric matrix A is positive definite if its eigenvalues are all strictly greater than zero, ensuring:

$$x^T A x > 0$$

To generate deterministic cubature points, the CKF requires calculating the “square root” of the P matrix. Because P is symmetric and positive definite, we use Cholesky Decomposition. This algorithm factors the matrix into the product of a lower triangular matrix L and its transpose L^T :

$$P = LL^T$$

The Engineering Dilemma

In pure mathematical simulations, covariance matrices remain perfectly positive definite. However, when deployed on embedded hardware, 32-bit floating-point truncation, asynchronous telemetry latency, and sudden injections of measurement noise can cause the P matrix to become slightly asymmetric or develop small negative eigenvalues. If this happens, a standard Cholesky solver will throw a NaN error and crash the tracking thread.

C++ Implementation: Robust Cholesky Decomposition

A production-grade tracking orchestrator must wrap the decomposition in a protective layer, enforcing symmetry and regularizing the diagonal to guarantee execution.

```
#include <Eigen/Dense>
#include <iostream>
#include <stdexcept>

using namespace Eigen;

// Computes the lower triangular matrix L of a covariance matrix P.
MatrixXd computeRobustCholesky(MatrixXd P) {
    // 1. Enforce strict mathematical symmetry to mitigate floating-point drift
    P = 0.5 * (P + P.transpose());

    // 2. Attempt standard Cholesky Decomposition
    LLT<MatrixXd> lltOfP(P);

    if (lltOfP.info() == Eigen::NumericalIssue) {
        // Engineering Fallback: Matrix lost positive definiteness.
        // Inject a tiny process noise (epsilon) along the diagonal
        // to force eigenvalues back into positive territory.
        double epsilon = 1e-6;
        MatrixXd PReg = P + MatrixXd::Identity(P.rows(), P.cols()) * epsilon;

        LLT<MatrixXd> lltOfPReg(PReg);
        if (lltOfPReg.info() == Eigen::Success) {
            return lltOfPReg.matrixL();
        } else {
            throw std::runtime_error("Cholesky decomposition failed post-regularization.");
        }
    }
}
```

```

    }

    return lltofP.matrixL();
}

```

2.3 Matrix inversion lemma (Woodbury identity) and proofs.

The Mathematical Concept

Inverting a matrix is one of the most computationally expensive operations in linear algebra ($O(n^3)$ complexity). The Woodbury matrix identity dictates that the inverse of a rank-k correction of a matrix can be computed by doing a rank-k correction to the inverse of the original matrix:

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$$

Application in Target Tracking

The Kalman Gain (K) dictates how much the filter trusts the incoming visual measurement versus its own internal prediction. It is calculated as:

$$K = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}$$

The term $(HP_{k|k-1}H^T + R)$ is the Innovation Covariance matrix (S). If we are fusing dense data arrays (where the measurement vector is massive), directly inverting S will cause pipeline stalls. The Woodbury identity allows us to mathematically rearrange the equation so we only invert matrices equal to the size of the state vector, rather than the measurement vector.

Engineering Example

Imagine an architecture utilizing an 8-state kinematic model (X, Y, Z, Vx, Vy, Vz, Width, Height) but processing an array of 50 simultaneous pixel threshold measurements from a segmented infrared target. Inverting a 50×50 matrix using standard LU decomposition on an embedded GPU/CPU complex is expensive. Using the Woodbury identity, the tracking engine can mathematically bypass the 50×50 inversion and instead invert an 8×8 matrix, maintaining a strict 60fps processing throughput.

Chapter 3

Differential Equations & Kinematics

While Chapter 2 provided the linear algebra tools to handle uncertainty, tracking algorithms ultimately need something physical to calculate. We must mathematically describe how a target moves through space.

The core challenge of tracking is a mismatch of domains: target physics (gravity, thrust, drag) operate continuously without interruption, but our tracking processors and visual sensors operate discretely, capturing snapshots of the world at 60 frames per second or processing telemetry at 10Hz. This chapter details how we model the continuous world and safely discretize it for embedded hardware.

3.1 Ordinary Differential Equations (ODEs) and continuous-time system modeling.

The Mathematical Concept

In kinematics, we describe a system's state using a vector $\mathbf{x}(t)$. To understand where the target will be in the future, we must model its rate of change over time. This is expressed as an Ordinary Differential Equation (ODE).

For a non-linear continuous-time dynamic system, the state derivative is defined as:

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t), \mathbf{w}(t))$$

Where: - $\dot{\mathbf{x}}(t)$ is the rate of change of the state vector (e.g., velocity is the derivative of position, acceleration is the derivative of velocity). - $f(\dots)$ is the non-linear function describing the physical laws governing the target. - $\mathbf{u}(t)$ is the known control input (e.g., autopilot commands, if tracking our own vehicle). - $\mathbf{w}(t)$ is the continuous-time process noise, capturing unknown disturbances like wind gusts or unmodeled target intent.

Application in Target Tracking

If we are tracking an uncooperative target—like an evasive drone—we do not have access to its control inputs $\mathbf{u}(t)$. We must rely entirely on our kinematic assumptions. The ODE simplifies to

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t)) + \mathbf{w}(t)$$

For a simple 1D position (p) and velocity (v) model where velocity is assumed constant, the ODE system looks like this:

$$\begin{aligned}\dot{p} &= v \\ \dot{v} &= 0 + w\end{aligned}$$

The Engineering Dilemma

Computers do not understand continuous time. An edge processor cannot evaluate an integral from $t = 0$ to $t = \infty$. It only understands arrays of memory evaluated at specific, discrete clock cycles. To predict a target's position at the time of the next camera shutter, we must numerically integrate the continuous ODE over a specific time step, Δt .

3.2 Discretization of continuous-time models (Runge-Kutta methods).

The Mathematical Concept

To convert continuous physics into discrete software updates ($\mathbf{x}_k \rightarrow \mathbf{x}_{k+1}$), we must approximate the integral of the ODE over the time interval Δt .

The simplest approach is the Euler method: $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta t \cdot f(\mathbf{x}_k)$. However, Euler integration assumes the rate of change remains completely flat over the entire Δt window. For high-speed targets executing turns, this assumption accumulates massive errors. Because the local error (error per step) is proportional to h^2 , and the global error is proportional to h , therefore it relies entirely on a linear approximation, which means it drifts significantly if the true curve is highly non-linear. For stiff equations or highly dynamic systems, if the step size h is not infinitesimally small, the Euler method can become numerically unstable, causing the state estimates to diverge wildly.

The gold standard for aerospace tracking is the 4th-Order Runge-Kutta (RK4) method. RK4 evaluates the continuous derivative at four distinct points across the Δt window (the beginning, two at the midpoint, and one at the end) and computes a weighted average.

The RK4 update equations are:

$$\begin{aligned}k_1 &= f(\mathbf{x}_k) \\ k_2 &= f\left(\mathbf{x}_k + \frac{\Delta t}{2}k_1\right) \\ k_3 &= f\left(\mathbf{x}_k + \frac{\Delta t}{2}k_2\right) \\ k_4 &= f(\mathbf{x}_k + \Delta tk_3) \\ \mathbf{x}_{k+1} &= \mathbf{x}_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)\end{aligned}$$

Engineering Example When dealing with asynchronous sensor telemetry, Δt is rarely a static value. If a camera frame drops, the time since the last measurement might jump from 16ms to 32ms. Integrating over this variable Δt using Euler would cause the predicted bounding box to jitter wildly. Using RK4,

the tracking engine safely interpolates the curved kinematic trajectory regardless of the fluctuating time step, ensuring the predicted bounding box lands exactly where the visual AI expects it.

C++ Implementation: The RK4 Integrator

In our object-oriented architecture, we isolate the integrator so it can be used by any kinematic model. Because we are operating on edge hardware where cycle counts matter, we pass the derivative function as a lambda or `std::function` to keep the memory footprint tight.

```
#include <Eigen/Dense>
#include <functional>

using namespace Eigen;

class KinematicIntegrator {
public:
    // f: The continuous-time ODE function describing the physics
    // xVector: The current state vector
    // dt: The exact time elapsed since the last measurement
    static VectorXd rungeKutta4(const std::function<VectorXd(const VectorXd&)>& f,
                                const VectorXd& xVector,
                                double dt) {

        VectorXd k1 = f(xVector);
        VectorXd k2 = f(xVector + 0.5 * dt * k1);
        VectorXd k3 = f(xVector + 0.5 * dt * k2);
        VectorXd k4 = f(xVector);

        return xVector + (dt / 6.0) * (k1 + 2.0*k2 + 2.0*k3 + k4);
    }
};
```

3.3 Modeling constant velocity (CV), constant acceleration (CA), and coordinated turn (CT) dynamics.

Once we have our integrator, we must define the physical functions ($f(x)$) we intend to track. Within an Interacting Multiple Model (IMM) architecture, we run several of these concurrently.

3.3.1 The Constant Velocity (CV) Model

The CV model assumes the target travels in a straight line at a constant speed. Any changes in velocity (accelerations) are modeled purely as random process noise.

For visual tracking applications, we utilize an 8-state vector: $[X, Y, Z, V_x, V_y, V_z, W, H]^T$, where W and H are the physical width and height of the target.

In continuous time, the derivative function $f(x)$ is simply the velocities:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$

- $\dot{Z} = V_z$
- $\dot{V}_x = 0$
- $\dot{V}_y = 0$
- $\dot{V}_z = 0$
- $\dot{W} = 0$ (Target size is physically constant).
- $\dot{H} = 0$ (Target size is physically constant)

Because the CV model is perfectly linear, it does not strictly require RK4 integration. The discrete-time kinematic equations become:

$$X_{k+1} = X_k + V_{x,k} \Delta t$$

$$V_{x,k+1} = V_{x,k}$$

$$W_{k+1} = W_k$$

$$H_{k+1} = H_k$$

Mapping these equations across all variables yields our highly efficient, 8×8 discrete-time State Transition Matrix (F_{CV}):

$$F_{CV} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The Engineering Dilemma: The Lag vs. Noise Trade-off For a vision-and-telemetry-based tracker, the CV model is incredibly powerful because it naturally filters out the high-frequency “pixel jitter” generated by neural network bounding boxes. By refusing to explicitly calculate acceleration, the filter prevents the target’s 3D state from vibrating wildly.

However, this mathematical stubbornness introduces a severe engineering dilemma: **Dynamic Lag**.

If an evasive target suddenly executes a sharp, high-G maneuver, the CV model mathematically assumes the target is still flying straight. It will confidently predict the target’s next position along that straight line. By the time the visual measurement proves the target has turned, the prediction error (the Innovation) becomes massive. If the filter trusts its internal physics too much, the bounding box will lag behind the physical target, eventually separating entirely and causing a dropped track.

To solve this without reverting to the noisy Constant Acceleration model, engineers use a mathematical workaround: Process Noise Inflation. Within the IMM architecture, we deploy a secondary “High-Agility” CV model. This model uses the exact same F_{CV} matrix, but couples it with a massively inflated Process Noise Covariance matrix (Q). When a maneuver occurs, the IMM shifts probability to this high-agility model. The inflated Q matrix forces the Kalman Gain to mathematically distrust its own straight-line prediction and snap directly to the raw visual measurements until the turn is complete, completely eliminating dynamic lag without adding matrix dimensionality.

C++ Implementation

Because the CV model is the baseline state for the majority of the tracking lifecycle, its matrix generation must be incredibly lean. In C++, constructing this 8×8 identity matrix and injecting the Δt parameters is highly cache-friendly.

```
#include <Eigen/Dense>

using namespace Eigen;

class KinematicModels {
public:
    // Generates the 8x8 State Transition Matrix for the Constant Velocity (CV) Model
    // dt: The time elapsed since the last update
    static MatrixXd getConstantVelocityMatrix(double dt) {
        // Initialize an 8x8 Identity matrix
        MatrixXd F = MatrixXd::Identity(8, 8);

        // Position updates linearly based on velocity (e.g., X = X + Vx * dt)
        F(0, 3) = dt;
        F(1, 4) = dt;
        F(2, 5) = dt;

        // Note: Velocities (indices 3, 4, 5) and Bounding Box dimensions
        // (indices 6, 7) inherently multiply by 1.0 from the Identity matrix,
        // representing their constant state.

        return F;
    }
};
```

3.3.2 The Constant Acceleration (CA) Model

The CA model introduces spatial acceleration states, expanding to an 11-state vector: $[X, Y, Z, V_x, V_y, V_z, A_x, A_y, A_z, W, H]$

The ODE becomes:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$
- $\dot{Z} = V_z$
- $\dot{V}_x = A_x$
- $\dot{V}_y = A_y$
- $\dot{V}_z = A_z$
- $\dot{A}_x = 0$
- $\dot{A}_y = 0$
- $\dot{A}_z = 0$
- $\dot{W} = 0$ (Target size is physically constant)
- $\dot{H} = 0$ (Target size is physically constant)

Therefore, the kinematic equations are:

- Position: $X_{k+1} = X_k + V_{x,k}\Delta t + \frac{1}{2}A_{x,k}\Delta t^2$

- Velocity: $V_{x,k+1} = V_{x,k} + A_{x,k} \Delta t$
- Acceleration: $A_{x,k+1} = A_{x,k}$

Mapping these equations across all spatial axes yields our 11×11 discrete-time State Transition Matrix (F_{CA}):

$$F_{CA} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & \frac{1}{2}\Delta t^2 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The Engineering Reality of CA: While theoretically rigorous for radar tracking ballistic missiles, the 11-state CA model is explicitly dangerous for vision-based tracking. Because visual AI bounding boxes suffer from high-frequency pixel jitter, an explicit acceleration state will interpret this jitter as violent, frame-to-frame G-forces. The filter will amplify this noise, causing the track to vibrate out of control. This is why our IMM architecture relies on an 8-state CV model with dynamically inflated process noise to simulate maneuvers, rather than explicitly tracking acceleration.

**** C++ Implementation for the CA Model:****

```
#include <Eigen/Dense>

using namespace Eigen;

class ConstantAccelerationModel {
public:
    // Generates the 11x11 State Transition Matrix for the CA Model
    static MatrixXd getTransitionMatrix(double dt) {
        MatrixXd F = MatrixXd::Identity(11, 11);

        double dt2Half = 0.5 * dt * dt;

        // Position updates based on velocity
        F(0, 3) = dt; F(1, 4) = dt; F(2, 5) = dt;

        // Position updates based on acceleration
        F(0, 6) = dt2Half; F(1, 7) = dt2Half; F(2, 8) = dt2Half;

        // Velocity updates based on acceleration
        F(3, 6) = dt; F(4, 7) = dt; F(5, 8) = dt;

        return F;
    }
};
```

```
}
};
```

3.3.3 The Coordinated Turn (CT) Model

The CT model is essential for tracking fixed-wing aircraft, which cannot simply slide sideways in the air; they must bank to turn. This introduces a Turn Rate parameter (ω). Assume that the tracking architecture relies on visual AI bounding boxes, you must also maintain the physical width and height of the target to utilize the scale prior. Adapting the CT model for your specific IMM-CKF pipeline results in a 9-state vector:

$$\mathbf{x} = [X, Y, Z, V_x, V_y, V_z, \omega, W, H]^T$$

- Z, V_z : Altitude and vertical velocity.
- W, H : Physical width and height of the target.

The standard CT model assumes the aircraft is banking to execute a turn in the horizontal plane ($X-Y$). Therefore, the turn rate ω strictly cross-couples and rotates the horizontal velocities (V_x and V_y). The vertical trajectory (Z -axis) is considered mathematically decoupled from the horizontal turn.

This model is highly non-linear. The ODE for a 3D horizontal turn is:

- $\dot{X} = V_x$
- $\dot{Y} = V_y$
- $\dot{Z} = V_z$
- $\dot{V}_x = -\omega V_y$
- $\dot{V}_y = \omega V_x$
- $\dot{V}_z = 0$
- $\dot{\omega} = 0$
- $\dot{W} = 0$
- $\dot{H} = 0$

Because the turn rate (ω) multiplies with the velocities (V_x and V_y), the Coordinated Turn model is non-linear. Therefore, a static F_{CT} matrix of pure constants does not exist. (In an EKF, you would derive the Jacobian matrix here, but because we are building a CKF, we do not need to linearize it).

Instead, we directly evaluate the closed-form discrete transition function $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \Delta t)$. The exact geometric equations for a target banking in the horizontal plane over time Δt are:

$$X_{k+1} = X_k + \frac{V_{x,k}}{\omega_k} \sin(\omega_k \Delta t) - \frac{V_{y,k}}{\omega_k} (1 - \cos(\omega_k \Delta t))$$

$$Y_{k+1} = Y_k + \frac{V_{x,k}}{\omega_k} (1 - \cos(\omega_k \Delta t)) + \frac{V_{y,k}}{\omega_k} \sin(\omega_k \Delta t)$$

$$V_{x,k+1} = V_{x,k} \cos(\omega_k \Delta t) - V_{y,k} \sin(\omega_k \Delta t)$$

$$V_{y,k+1} = V_{x,k} \sin(\omega_k \Delta t) + V_{y,k} \cos(\omega_k \Delta t)$$

(Note: Z , V_z , ω , W , and H remain linearly constant or are integrated straight forwardly).

The Engineering Dilemma (Division by Zero): Notice that the equations for X and Y require dividing by ω . If the aircraft stops turning and flies straight, ω approaches 0, which will cause a divide-by-zero hardware exception (NaN) and crash your filter. To solve this, your C++ implementation must check if ω is near zero, and if so, safely fall back to the standard Constant Velocity calculation for that frame.

C++ Implementation for the CT Model: In the CKF, you will pass your 18 cubature points (2×9 states) through this exact function during the Prediction Step.

```
#include <Eigen/Dense>
#include <cmath>

using namespace Eigen;

class CoordinatedTurnModel {
public:
    // Evaluates the non-linear discrete state transition for the CT model.
    // xState: The 9-state vector [X, Y, Z, Vx, Vy, Vz, omega, W, H]^T
    static VectorXd evaluateTransition(const VectorXd& xState, double dt) {
        VectorXd xNext = xState; // Initialize with current state to carry over constants (Z, Vz, w, W, H)

        double vx = xState(3);
        double vy = xState(4);
        double w = xState(6);

        // Z position updates linearly based on Vz
        xNext(2) = xState(2) + xState(5) * dt;

        // Engineering Check: Prevent divide-by-zero if the target is flying straight
        if (std::abs(w) < 1e-5) {
            // Degenerate to Constant Velocity model for X and Y
            xNext(0) = xState(0) + vx * dt;
            xNext(1) = xState(1) + vy * dt;

            // Velocities remain constant
            xNext(3) = vx;
            xNext(4) = vy;
        } else {
            // Apply the non-linear Coordinated Turn equations
            double sin_w_dt = std::sin(w * dt);
            double cos_w_dt = std::cos(w * dt);

            xNext(0) = xState(0) + (vx / w) * sin_w_dt - (vy / w) * (1.0 - cos_w_dt);
            xNext(1) = xState(1) + (vx / w) * (1.0 - cos_w_dt) + (vy / w) * sin_w_dt;

            xNext(3) = vx * cos_w_dt - vy * sin_w_dt;
            xNext(4) = vx * sin_w_dt + vy * cos_w_dt;
        }
    }
};
```

```
    }  
    return xNext;  
}  
};
```

Chapter 4

Probability Theory & Stochastic Processes

- 4.1 Random variables, probability density functions (PDFs), and Bayes' Theorem.**
- 4.2 Gaussian distributions and their properties (multivariate normal distribution).**
- 4.3 Markov processes and transition probabilities.**

Part II: Dynamic Systems and The Optimal Estimator

Chapter 5

Control System Fundamentals

5.1 State-space representation of dynamic systems.

5.2 Observability and controllability.

5.3 Process noise vs. measurement noise.

Chapter 6

The Bayesian Filtering Framework

6.1 The recursive Bayes filter: Prediction and Update steps.

6.2 Why closed-form solutions are rare (the integration problem).

Part III: The Classic Standard Kalman Filter (KF)

Chapter 7

Derivation of the Standard KF

7.1 Mathematical proof of the linear Kalman Filter from Bayesian principles.

7.2 The Minimum Mean Square Error (MMSE) estimator.

Chapter 8

Tuning and Diagnostics

8.1 Covariance matching and innovation analysis.

8.2 Filter divergence and numerical stability considerations.

Part IV: Conquering Non-Linearity

Chapter 9

The Extended Kalman Filter (EKF)

9.1 Linearizing non-linear functions using the Jacobian.

9.2 Mathematical proof and limitations of the EKF (truncation errors).

Chapter 10

The Unscented Kalman Filter (UKF)

10.1 The Unscented Transform (UT): Choosing sigma points.

10.2 Proof of how UT captures mean and covariance better than linearization.

Chapter 11

The Cubature Kalman Filter (CKF)

11.1 The spherical-radial cubature rule.

11.2 Mathematical derivation of cubature points.

11.3 Comparative analysis: CKF vs. UKF in high-dimensional state spaces.

Part V: Maneuvering Targets and Multiple Models

Chapter 12

Interacting Multiple Model (IMM) Estimation

12.1 The challenge of maneuvering target tracking.

12.2 Markov chain mixing of model probabilities.

12.3 The IMM architecture: Interaction, Filtering, Model Probability Update, and Combination.

Chapter 13

The Synthesis: IMM-CKF

- 13.1 Embedding the Cubature Kalman Filter within the IMM framework.**
- 13.2 Handling highly non-linear maneuvers (e.g., transitioning from a linear trajectory to a sharp, high-G coordinated turn).**
- 13.3 Complete algorithm walkthrough and mathematical proof of convergence**

Part VI: Architecture and Implementation

Chapter 14

Software Architecture for State Estimation

- 14.1 Object-oriented design patterns for filter development (e.g., in modern C++).**
- 14.2 Memory management and real-time processing constraints.**

Chapter 15

Hardware Deployment Considerations

15.1 Deploying complex estimators on embedded systems.

15.2 Exploiting parallelization for matrix operations.

Chapter 16

Bibliography

16.1 Articles

[1] Arasaratnam, I., & Haykin, S. (2009). *Cubature Kalman Filters*. IEEE Transactions on Automatic Control.

16.2 Books

[2] Simon, D. (2006). *Optimal State Estimation: Kalman, H^∞ , and Nonlinear Approaches*. John Wiley & Sons.

Bibliography

Index